

Aggregate vs Window Function Comparison Reference

Aggregate vs Window Function Comparison Reference

Covers Chapters 6 and 11 — Keep this open whenever you write analytical queries.

The Core Distinction

Behaviour	GROUP BY Aggregate	Window Function
Collapses rows into one per group?	Yes	No
Preserves every original row?	No	Yes
Requires OVER clause?	No	Yes
Can filter results with HAVING?	Yes	No (use outer WHERE)
Enables row-to-row comparison?	No	Yes
Supports ranking within a group?	No	Yes

Side-by-Side Syntax: The Same Question, Two Approaches

Total Sales per Region

GROUP BY Aggregate — one output row per region:

```
SELECT region, SUM(sale_amount) AS total_sales
FROM orders
GROUP BY region;
```

Window Function — every row kept, running total alongside it:

```
SELECT order_id, region, sale_amount,
       SUM(sale_amount) OVER (PARTITION BY region) AS total_sales
FROM orders;
```

Key insight from Chapter 11: PARTITION BY region inside OVER() does the same segmenting work as GROUP BY region, but the individual rows — order_id, sale_amount — remain fully visible in every output row.

Function-by-Function Equivalents

Goal	GROUP BY Version	Window Function Version
Count rows per category	COUNT(*) ... GROUP BY category	COUNT(*) OVER (PARTITION BY category)
Sum values per region	SUM(amount) ... GROUP BY region	SUM(amount) OVER (PARTITION BY region)
Average per department	AVG(salary) ... GROUP BY dept	AVG(salary) OVER (PARTITION BY dept)
Minimum value per group	MIN(price) ... GROUP BY group	MIN(price) OVER (PARTITION BY group)
Maximum value per group	MAX(score) ... GROUP BY group	MAX(score) OVER (PARTITION BY group)
Rank rows within a group	Not possible directly	RANK() OVER (PARTITION BY group ORDER BY score DESC)
Running total over time	Not possible directly	SUM(amount) OVER (PARTITION BY region ORDER BY sale_date)
Compare row to previous row	Not possible directly	LAG(amount) OVER (PARTITION BY region ORDER BY sale_date)

WHERE vs HAVING vs Windowed Filtering

- WHERE — filters individual rows before aggregation (Chapter 6). Works with both aggregate queries and window function queries.
- HAVING — filters after GROUP BY aggregation; applies to the collapsed result. Not available with window functions.
- Outer WHERE on a subquery — the correct way to filter on a window function result:

```
SELECT * FROM (  
    SELECT employee_id, dept,  
           RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dept_rank  
    FROM employees  
    ) ranked  
WHERE dept_rank = 1;
```

When to Reach for Each Tool

Use GROUP BY aggregates when you need:

- A summary table — one row per category, region, or time period
- Totals, averages, or counts to report at the group level
- HAVING to exclude low-volume or low-value groups from results

Use window functions when you need:

- Every detail row plus an aggregate value side by side

- Rankings, sequences, or numbered rows within a group (ROW_NUMBER, RANK, DENSE_RANK)
 - Row-to-row comparisons such as month-over-month change (LAG, LEAD)
 - Running totals or moving averages that accumulate across ordered rows
-

NULL Awareness (Chapter 6 Reminder)

- COUNT(*) counts every row including NULLs.
 - COUNT(column_name) counts only non-NULL values in that column.
 - SUM, AVG, MIN, and MAX all silently ignore NULLs.
 - This behaviour carries over identically when these functions are used inside OVER().
-

Quick Syntax Template

```
-- Aggregate
SELECT <group_col>, <AGG_FUNC>(<col>) AS alias
FROM table
[WHERE condition]
GROUP BY <group_col>
[HAVING <AGG_FUNC>(<col>) condition];

-- Window
SELECT <cols>,
       <AGG_FUNC | RANK | LAG | LEAD>(<col>)
       OVER (PARTITION BY <group_col> ORDER BY <order_col>) AS alias
FROM table
[WHERE condition];
```

Chapter 6 builds the aggregation foundation. Chapter 11 extends it — once you know both, you choose the right tool instinctively.